

EXPERIMENT 3

Push a Docker Image to Docker Hub

Objective

To learn how to take an existing Docker image created in Experiment 1, authenticate with Docker Hub, tag the image with a Docker Hub repository name, push the image to Docker Hub, and verify that the image is available in the online registry.

Learning Outcomes

- Check and use an existing Docker image.
- Create and use a Docker Hub account.
- Authenticate Docker from PowerShell.
- Tag a local image using the Docker Hub repository naming format.
- Push the image to Docker Hub.
- Verify the uploaded repository and image tag.
- Understand how Docker Hub prepares an image for use on another machine.

Requirements

- Windows 10 or Windows 11 computer.
- Docker Desktop installed and running.
- PowerShell.
- The Docker image my-python-app created in Experiment 1.
- A Docker Hub account.
- Internet connection for authentication and image upload.

Important: In this experiment we do not rebuild the Python application. We use the existing image my-python-app and publish it to Docker Hub. Replace USERNAME in the commands with your own Docker Hub username.

PART 1 - START DOCKER

Step 1: Open Docker Desktop

Open the Windows Start menu, search for Docker Desktop, open it, and wait until Docker finishes starting.

Open Docker Desktop

Start Menu
↓
Docker Desktop

- Docker Desktop provides the local Docker environment.
- The Docker engine must be running before Docker commands are used.
- PowerShell is used to enter the commands in this experiment.

Step 1 continued: Check Docker

Open PowerShell and check the installed Docker version.

Run in PowerShell

```
docker --version
```

Expected result

```
Docker version 29.x.x
```

- This confirms that the Docker command is available.
- The exact version may be different on different computers.
- A valid version allows the experiment to continue.

PART 2 - CHECK THE EXISTING IMAGE

Step 2: Display Docker images

Run the following command to check whether the image from Experiment 1 is present.

Run in PowerShell

```
docker images
```

Expected result

```
REPOSITORY TAG IMAGE ID  
my-python-app latest xxxxxxxxxxxx
```

- The image my-python-app was created in Experiment 1.
- We do not run docker build again in this experiment.
- The image must exist locally before it can be tagged and pushed.

PART 3 - CREATE A DOCKER HUB ACCOUNT

Step 3: Open Docker Hub in a web browser

Open Google Chrome or Microsoft Edge. Search for Docker Hub and open the official Docker Hub website.

Browser search

```
Docker Hub
```

- Docker Hub is an online registry for storing and sharing Docker images.
- It is accessed through a web browser; there is no separate Docker Hub application to install for this lab.
- Students need a Docker Hub account so they can upload their image.

Step 4: Create or sign in to the Docker Hub account

Choose Sign Up if you do not already have an account. If an account already exists, sign in.

Example account information

```
Docker Hub username: USERNAME
```

- The Docker Hub username becomes part of the image name.
- For example, the repository name can be USERNAME/my-python-app.
- Students should use their own Docker Hub username in later commands.

PART 4 - LOGIN FROM POWERSHELL

Step 5: Log in to Docker Hub

Return to PowerShell and run the Docker login command.

Run in PowerShell

```
docker login
```

Expected result

```
Login Succeeded
```

- Docker authenticates the local Docker CLI with Docker Hub.
- The message may indicate that existing credentials are being used.
- Authentication is required before the image can be pushed.

If Docker is already logged in, you may see a message similar to Authenticating with existing credentials... followed by Login Succeeded.

PART 5 - UNDERSTAND THE DOCKER HUB IMAGE NAME

Step 6: Build the image name

Docker Hub images are identified using a repository name that includes the Docker Hub username.

Naming format

```
USERNAME/my-python-app:v1
```

- USERNAME is your Docker Hub username.
- my-python-app is the repository name.
- v1 is the image tag identifying the version.
- The final name tells Docker where the image belongs when it is pushed.

Example

```
dheerajj1/my-python-app:v1
```

Use your own Docker Hub username rather than the example username.

PART 6 - TAG THE EXISTING IMAGE

Step 7: Tag the local image

Give the existing my-python-app image a Docker Hub-compatible name. Replace USERNAME with your Docker Hub username.

Run in PowerShell

```
docker tag my-python-app USERNAME/my-python-app:v1
```

- docker tag does not rebuild the application.
- It gives the existing image another name and tag.
- The tag connects the local image to the Docker Hub repository name.
- The version tag v1 identifies this upload as version 1.

Step 8: Check the image tags

List the images again and check that the new Docker Hub-style name appears.

Run in PowerShell

```
docker images
```

Expected result

```
REPOSITORY TAG IMAGE ID
my-python-app latest xxxxxxxxxxxx
USERNAME/my-python-app v1 xxxxxxxxxxxx
```

- Both names can refer to the same underlying image.
- The image ID should be the same for the local name and the tagged name.
- Tagging creates a new reference, not a new build.

Important idea

```
Same Docker Image
|
├── my-python-app:latest
└── USERNAME/my-python-app:v1
```

PART 7 - PUSH THE IMAGE TO DOCKER HUB

Step 9: Push the image

Upload the tagged image to Docker Hub. Replace USERNAME with your Docker Hub username.

Run in PowerShell

```
docker push USERNAME/my-python-app:v1
```

Expected result

```
The push refers to repository [docker.io/USERNAME/my-python-app]
...
v1: digest: sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx size: xxx
```

- Docker uploads the image layers to the Docker Hub repository.
- You may see several layers reported as Pushed.
- The exact digest, layer identifiers, and output size are different for different images and versions.
- A successful push means the image has been stored in Docker Hub.

PART 8 - VERIFY THE IMAGE ON DOCKER HUB

Step 10: Open your repository

Open Docker Hub in the browser, sign in, and open your repository named my-python-app.

Repository structure

```
Docker Hub
↓
USERNAME
↓
my-python-app
↓
v1
```

- The repository should belong to the student's Docker Hub account.
- The v1 tag should be visible for the uploaded image.
- This confirms that the image has been transferred from the local computer to the remote registry.

Step 11: Understand what is now available

Compare the local image and the Docker Hub image.

Local

```
my-python-app:latest
```

Remote

```
USERNAME/my-python-app:v1
```

- The local computer still keeps the image.
- Docker Hub now stores the tagged version remotely.
- Another Docker-enabled machine can later download the image using its Docker Hub name.

PART 9 - UNDERSTAND THE COMPLETE PUSH PROCESS

Before Experiment 3

```
Student Computer
↓
Docker Image
my-python-app:latest
```

After Experiment 3

```
Student Computer
|
| docker tag
↓
USERNAME/my-python-app:v1
|
| docker push
↓
Docker Hub
↓
USERNAME/my-python-app:v1
```

The image is now stored both locally and in Docker Hub. The Docker Hub copy can be obtained by another Docker-enabled machine in the next experiment.

IMAGE BUILD vs TAG vs PUSH

docker build

```
docker build -t my-python-app .
```

Creates a Docker image from the Dockerfile and project files.

docker tag

```
docker tag my-python-app USERNAME/my-python-app:v1
```

Gives the existing image another name/tag suitable for a Docker Hub repository.

docker push

```
docker push USERNAME/my-python-app:v1
```

Uploads the tagged image to Docker Hub.

```
docker build
↓
CREATE IMAGE

docker tag
↓
GIVE IMAGE A REGISTRY NAME

docker push
↓
UPLOAD IMAGE
```

COMPLETE EXPERIMENT 3 FLOW

```
Existing Docker Image
my-python-app:latest
|
↓
docker login
|
↓
Docker Hub Authentication
|
↓
docker tag
|
↓
USERNAME/my-python-app:v1
|
↓
docker push
|
↓
Docker Hub
|
↓
Image Stored Online
```

LOCAL IMAGE AND REMOTE IMAGE

```
LOCAL COMPUTER
|
└─ my-python-app:latest

DOCKER HUB
|
└─ USERNAME/my-python-app:v1
```

The local image is the image used to create containers on the student's computer. The Docker Hub repository is the remote location from which another Docker-enabled computer can obtain the image.

PROJECT DESCRIPTION - STEP-BY-STEP UNDERSTANDING

- 1. Starting point: The experiment begins with the my-python-app image created in Experiment 1. The Python application, Flask dependency, and Docker environment have already been packaged into this image.
- 2. Docker Hub account: Students create or use an existing Docker Hub account. Docker Hub provides an online registry where container images can be stored and shared.
- 3. Authentication: The docker login command authenticates the local Docker CLI with Docker Hub. Without successful authentication, the student cannot push an image to the account's repository.
- 4. Repository naming: Docker Hub requires the image to be identified using the student's username and repository name. The example format is USERNAME/my-python-app:v1. The

username identifies the Docker Hub account, while v1 identifies the image version.

- 5. Tagging: The docker tag command does not rebuild the application and does not create a separate build process. It creates another name for the existing image so Docker knows which Docker Hub repository and tag will be used.
- 6. Comparing image IDs: After tagging, the local name my-python-app:latest and the Docker Hub-style name USERNAME/my-python-app:v1 can show the same Image ID. This demonstrates that both names refer to the same underlying image.
- 7. Pushing: The docker push command uploads the required image layers to Docker Hub. Docker may display each layer as it is pushed and then report a digest for the uploaded image.
- 8. Verification: Students sign in to Docker Hub and open the my-python-app repository. The v1 tag should be visible, confirming that the image is stored in the remote registry.
- 9. Local and remote copies: After a successful push, the image remains on the student's computer and is also available in Docker Hub. The remote copy can be retrieved later from another Docker-enabled machine.
- 10. Why this matters: Experiment 1 created a containerized application on one computer. Experiment 3 adds a way to distribute the image. This is the bridge between local containerization and deployment on another machine.
- 11. Preparation for Experiment 4: In the next experiment, another computer or server can use docker pull USERNAME/my-python-app:v1 to obtain the image from Docker Hub and then create a container from it.
- 12. Overall concept: The complete workflow is local image → login → tag → push → Docker Hub → pull on another machine. Docker Hub acts as the shared registry between the machines.

KEY COMMANDS USED

```
docker --version
docker images
docker login
docker tag my-python-app USERNAME/my-python-app:v1
docker images
docker push USERNAME/my-python-app:v1
```

Expected final result: The student successfully tags the existing my-python-app image as USERNAME/my-python-app:v1, pushes it to Docker Hub, and verifies the repository and v1 tag online.